

Name : Neil Taison RIGAUD

Student ID : 411221367

Subject : Intro to Image Processing – Assignment #2

Date : 2025/10/22

1- Code Listing and Explanation

```
28 import numpy as np
29 import matplotlib.pyplot as plt
30 from PIL import Image
```

We import the necessary libraries for:

- Opening and saving images:
 - o **Image** from the **PIL** library
- Processing the pixel values from the opened images:
 - o **numpy**
- Creating the histograms:
 - o **pyplot** from **matplotlib**

```
32 # Constants
33 RGB_WEIGHTS = [0.299, 0.587, 0.114]
34 IMG_TITLE = "lena.png"
35 OUTPUT_TITLE = "lena_converted.png"
```

We define the weights for converting the color image to a grayscale using the given formula in the previous assignment along with constant values for opening and saving image file outputs.

```

38 def color_to_gray(img_title, output_title):
39     img = Image.open(img_title)
40
41     # Get the pixels data
42     pixels = np.array(img)
43
44     # If image has alpha channel, ignore it
45     if pixels.ndim == 3 and pixels.shape[2] >= 3:
46         # Compute weighted sum across color channels -> single channel gray
47         # Use dot product with RGB_WEIGHTS and clip to uint8
48         gray = np.dot(pixels[..., :3], np.array(RGB_WEIGHTS))
49     else:
50         # Already single channel
51         gray = pixels.astype(float)
52
53     # Convert to uint8
54     gray_uint8 = np.clip(gray, 0, 255).astype(np.uint8)
55
56     # Create a 3-channel image where all channels equal gray for saving
57     gray_rgb = np.stack([gray_uint8] * 3, axis=-1)
58
59     # Save the output grayscale image (visualized as RGB where channels equal)
60     out_img = Image.fromarray(gray_rgb)
61     out_img.save(output_title)
62
63     return gray_uint8
64

```

Here we proceed to convert the color image to a grayscale image by using the conversion formula with the previously defined weights to give each of the RGB channels the same weighted sum value for each pixels in the image.

```

66 def count_gray_levels(pixels, range_max=256):
67     """Count the gray level values in an array of pixels values."""
68     # Flatten and use numpy bincount for speed and correctness
69     flat = pixels.ravel()
70     obs = np.bincount(flat, minlength=range_max)
71     return obs
72

```

This function is used to count the number of observed gray level values in the image for the range 0 ~ 255.

```

94 def histogram_equalization(gray_levels):
95     """
96     Given gray_levels (counts per original gray level 0..L-1),
97     return:
98     - equalized_counts: counts per equalized gray level (0..L-1) after mapping
99     - mapping: array of length L where mapping[k] is the new level for original level k
100    """
101    L = len(gray_levels)
102    cumsum = np.cumsum(gray_levels)
103    n = cumsum[-1]
104
105    # Avoid division by zero
106    if n == 0:
107        return np.zeros(L), np.arange(L)
108
109    # mapping (s[k]) = round((L-1)/n * cumulative_sum[k])
110    mapping = np.round((L - 1) * cumsum / n).astype(int)
111
112    # Aggregate pixel counts into mapped bins
113    equalized_counts = np.bincount(mapping, weights=gray_levels, minlength=L)
114
115    return equalized_counts, mapping

```

This is the core function for the histogram equalization procedure where we start by computing the **cumulative summation** for each gray level value **i**, then we round the values previously obtained into a **mapping of rounded values** that will be used for generating the enhanced image. Finally we count the respective observations for each of the distinct **rounded values of the cumulative summations** and store them into **equalized_counts**. Finally we return those values for generating the resulting image and produce the equalized histogram.

```

118 def create_barplot(obs, x_label, y_label, title, output_name, edge_color="black"):
119     """Creates a barplot histogram for a list of observations."""
120
121     # Increasing array for the x-axis
122     x_values = np.arange(len(obs))
123
124     # Plot the histogram
125     plt.bar(
126         x_values, obs, width=0.2, align="center", color="skyblue", edgecolor=edge_color
127     )
128     plt.xlabel(x_label)
129     plt.ylabel(y_label)
130     plt.title(title)
131     plt.savefig(output_name, dpi=300, bbox_inches="tight")
132     plt.close()

```

This function helps create the different barplots that we need before and after the histogram equalization process. As mentioned earlier, we use **pyplot** to plot the **discrete histogram** from the array of gray level observations and save the resulting image using the name parameter passed.

```

74 def apply_mapping(gray_pixels, mapping, output_title):
75     """Apply the histogram-equalization mapping to a 2D gray image
    and save result.
76
77     mapping: 1D array where mapping[old_level] = new_level
78     Returns the mapped 2D uint8 image.
79     """
80     # Ensure mapping can index up to 255
81     mapping = np.asarray(mapping).astype(np.uint8)
82
83     # Apply mapping using vectorized indexing
84     mapped = mapping[gray_pixels]
85
86     # Save as 3-channel RGB for viewing
87     mapped_rgb = np.stack([mapped] * 3, axis=-1)
88     out_img = Image.fromarray(mapped_rgb)
89     out_img.save(output_title)
90
91     return mapped

```

This function uses a mapping generated from the computation of the **rounded cumulative sum** of the gray level values from the **Histogram Equalization** procedure and generates the resulting enhanced image after the transformation.

```

135 if __name__ == "__main__":
136     pixels = color_to_gray(IMG_TITLE, OUTPUT_TITLE)
137     gray_level_counts = count_gray_levels(pixels)
138
139     create_barplot(
140         gray_level_counts,
141         "Gray level value",
142         "Observations",
143         "Histogram BEFORE Equalization",
144         "before_equalization.png",
145         "blue",
146     )
147
148     equalized_counts, mapping = histogram_equalization(gray_level_counts)
149     create_barplot(
150         equalized_counts,
151         "Gray level value",
152         "Observations",
153         "Histogram AFTER Equalization",
154         "after_equalization.png",
155         "red",
156     )
157
158     # Apply mapping to produce equalized image
159     equalized_img = apply_mapping(pixels, mapping, "lena_equalized.png")

```

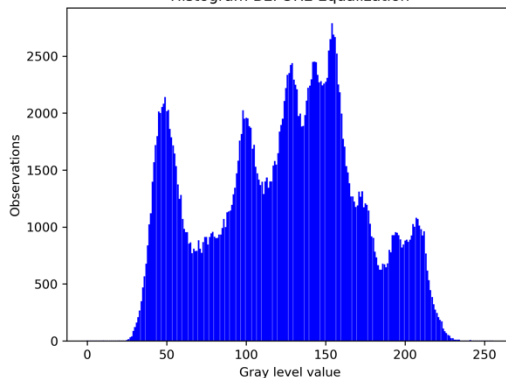
Finally, we have the main program procedure which uses the previously defined functions to:

1. Convert the original color image into a grayscale image using the ***color_to_gray*** function.
2. Obtain the gray level values for creating the original histogram for the converted grayscale image using the ***count_gray_levels*** function.
3. Create the histogram **before** equalization using ***create_barplot***
 - a. Note: We use *barplot* in the function name because we are using the *discrete* probability distribution.
4. Obtain the equalized gray level count values and mapping to generate the enhanced image and final equalized histogram.
5. Create the histogram **after** equalization using the equalized gray level values.

2- Result images and corresponding histograms



Histogram BEFORE Equalization



Histogram AFTER Equalization

